This is the html version of the file https://www.researchgate.net/profile/Praveen-Kumar-Koppanati/publication/387743058_Handling_Dynamic_Web_Elements_in_Selenium_for_Robust_Automation/links/677aed64894c55208545b44a/Handling-Dynamic-Web-Elements-in-Selenium-for-Robust-Automation.pdf.
Google automatically generates html versions of documents as we crawl the web.
These search terms have been highlighted: **handling dynamic web elements selenium** robust automation

Available online www.ejaet.com

European Journal of Advances in Engineering and Technology, 2021, 8(2):138-143

Research Article **ISSN: 2394 - 658X**

Handling Dynamic Web Elements in Selenium for Robust Automation

Praveen Kumar Koppanati

praveen.koppanati@gmail.com

ABSTRACT

The advent of modern **web** applications has introduced significant complexities into the domain of automation testing. **Web** applications now dynamically generate content based on user interactions and asynchronous server communications, often creating **elements** on-the-fly using frameworks such as Angular, React, or Vue.js. Consequently, traditional automation tools often fail to keep pace with these **dynamic web elements**, resulting in unstable and unreliable test cases. **Selenium**, a widely adopted tool for **web** automation, provides powerful capabilities for interacting with these **dynamic elements**, but it requires a strategic approach to fully handle their unpredictable nature.

This paper investigates various techniques for **handling dynamic web elements** within **Selenium**, offering a comprehensive guide to designing robust automation scripts. Key topics covered include the challenges posed by **dynamic elements**, strategies for using advanced locators such as XPath and CSS selectors, implementing explicit and fluent waits, **handling** asynchronous loading, and creating stable tests across multiple browsers. Additionally, we explore best practices for integrating **Selenium** with external libraries and frameworks to enhance the overall reliability and maintainability of automated testing.

Keywords: **Selenium**, **dynamic web elements**, **web** automation, asynchronous content, automation testing, robust automation, **dynamic** waits, cross-browser compatibility, XPath, CSS selectors, Page Object Model.

INTRODUCTION

The rise of **dynamic** and interactive **web** applications has revolutionized the way users interact with **web** content, providing seamless user experiences and rich interfaces. However, from an automation perspective, this evolution introduces a set of new challenges that require a new way of automation approach to testing. **Selenium** being an open-source tool for **web** automation, has emerged as the go-to solution for testers worldwide, given its flexibility, cross-browser support and adaptability to various programming languages. However, **handling dynamic web elements** and components that may not be immediately available in the Document Object Model (DOM) or those generated dynamically presents significant hurdles for achieving reliable test automation.

Traditional automation scripts, which rely on fixed locators or static page structures, struggle to interact with **elements** that change over time, are created after an asynchronous load, or have properties that vary depending on user interaction. Without a strategic approach, automation testers often face "flaky" tests, tests that pass or fail intermittently due to these **dynamic** behaviors rather than actual defects in the application under test.

This paper aims to provide a detailed exploration of techniques and methodologies for effectively **handling dynamic web elements** in **Selenium**. We will focus on leveraging advanced locators, waits, and integration strategies that ensure the reliability and robustness of automation scripts. The integration of these techniques will enable testers to handle the growing complexity of modern **web** applications while reducing the need for frequent test maintenance.

Fig. 1** Distribution of Automation Tools for **Dynamic Web Elements

BACKGROUND

Selenium's origins trace back to 2004, when it was first developed as an internal tool at ThoughtWorks to address limitations in existing **web** automation tools. It quickly grew in popularity due to its ability to automate user interactions with **web** browsers. **Selenium** allows for the simulation of real user actions such as clicking buttons, filling out forms, and navigating between pages across different browsers like Chrome, Firefox, Safari, and Internet Explorer. Its compatibility with multiple programming languages, including Java, Python, C#, and Ruby made it the perfect tool for automation testing and grew its popularity.

Despite its powerful capabilities, **Selenium's** core challenge remains the reliable interaction with **dynamic web elements**. **Dynamic** content, which is often loaded asynchronously via AJAX or other JavaScript-driven technologies, can be difficult to locate or even predict. Testing teams frequently encounter scenarios where an element is not present when the script runs, or its properties (such as ID or class name) change upon each page reload.

For example, in applications built with modern frameworks like React and Angular, content is often not fully loaded when the page initially renders. Instead, it is dynamically inserted or modified based on user actions, backend API responses, or even network conditions. This complexity requires more sophisticated strategies for locating, waiting, and interacting with **web elements** in order to create reliable and consistent test results.

CHALLENGES IN HANDLING DYNAMIC WEB ELEMENTS

***Fig. 2** Frequency of Common Issues in Automation Testing*

Dynamic Locators:

One of the core difficulties in interacting with **dynamic web elements** is the use of locators, the methods by which **Selenium** identifies and interacts with specific **elements** on a webpage. In static websites, attributes such as element ID, class, or tag name are usually consistent, making them ideal for locating **elements**. However, in **dynamic** applications, these attributes are often **dynamic** or auto generated, meaning they can change with every page load or user interaction.

For example, a **dynamic web** page may generate a unique ID for each user session, or the element's class may be determined based on JavaScript logic. This unpredictability requires more flexible locator strategies, such as using relative XPath expressions or CSS selectors that can adapt to changing attributes. XPath expressions allow testers to navigate the DOM and locate **elements** based on their relationship to other **elements**. CSS selectors, on the other hand, offer fast, performance-efficient options for locating **elements** based on their styling attributes.

Asynchronous Content:

AJAX (Asynchronous JavaScript and XML) has become a standard feature in **web** development, enabling pages to load content without refreshing. While this enhances user experience, it introduces challenges for writing automation scripts. A common problem occurs when the script tries to interact with an element before it is fully loaded, leading to exceptions like "element not found" or "stale element reference." **Selenium** needs to be able to wait for these **elements** to load and become interactable.

Asynchronous content also poses a challenge because there's no deterministic way to know exactly when content will finish loading. Varying server response times, network latency, and other factors all contribute to this uncertainty. Standard waits (i.e., having the script sleep for a fixed amount of time) are not ideal, as they can either cause unnecessary delays or still result in failures if the content takes longer than expected to load.

Cross-Browser Compatibility:

Another issue arises when testing **dynamic elements** across different browsers. Although **Selenium** is designed for cross-browser testing, differences in how browsers render **dynamic** content can result in varying test behavior. Some browsers, such as Chrome and Firefox, may handle asynchronous **elements** differently from Internet Explorer or Safari, leading to inconsistencies in how **elements** are located and interacted with.

For instance, an element that loads in 2 seconds on Chrome might take 3 seconds in Firefox or may not appear in the same location in Internet Explorer. These differences require careful tuning of test scripts to ensure they run consistently across all targeted browsers

Stale Element Reference:

A common exception encountered when dealing with **dynamic elements** is the "stale element reference" error. This occurs when an element is removed or altered in the DOM after it has been located by the script. Because **Selenium** holds a reference to the old version of the element, any subsequent interactions with it fail. This issue is particularly prevalent in **dynamic web** applications that update **elements** in response to AJAX calls or other asynchronous events.

STRATEGIES FOR HANDLING DYNAMIC WEB ELEMENTS IN SELENIUM

Advanced Locators:

To address the challenge of **dynamic** locators, **Selenium** supports a range of powerful locators that go beyond basic element attributes like ID, name, or class. Two of the most versatile are XPath and CSS selectors.

*Fig. 3 Error Reduction Over Test Iterations: Traditional vs **Selenium** Methods*

• **Relative XPath:** XPath allows testers to navigate through the structure of the DOM, locating **elements** based on their relationships with other **elements**. Unlike absolute XPath, which specifies a path starting from the root of the DOM, relative XPath allows the script to find **elements** based on their proximity to more stable **elements**. For example, if the parent element of a **dynamic** button has a consistent ID, the button can be located relative to this parent element using an XPath expression. This method provides flexibility in **handling elements** with **dynamic** IDs or classes, as it bypasses the need to rely on those attributes directly.
• **CSS Selectors:** CSS selectors are generally faster than XPath and can be more readable. They allow automation testers to locate **elements** based on their styling attributes, such as class or tag. CSS selectors can also match **elements** based on partial attribute values, making them ideal for **dynamic elements** that have auto-generated class names or IDs that follow a pattern. For example, if an element's ID includes a random string appended to a constant prefix, a CSS selector can match the constant part of the ID and ignore the random portion.

Explicit and Fluent Waits:

Selenium provides several mechanisms for waiting until an element is ready to be interacted with, which is crucial for **handling** asynchronous content.
• **Explicit Waits:** Explicit waits allow the test script to pause execution until a specified condition is met, such as an element becoming visible, clickable, or present in the DOM. By using explicit waits, testers can avoid timing issues

that arise when trying to interact with **elements** before they have fully loaded. **Selenium's** WebDriverWait class provides an implementation of explicit waits, enabling scripts to wait for a variety of conditions.
• **Fluent Waits:** Fluent waits are a more advanced form of explicit waits. They allow the script to periodically poll the DOM at regular intervals to check if the desired condition has been met. Fluent waits also offer the ability to handle exceptions during polling, such as retrying when a stale element reference exception is thrown. This makes fluent waits a powerful tool for **handling elements** that may disappear or become stale during asynchronous operations. For example, if an AJAX call updates an element in the background, causing a stale element reference, fluent waits can catch this and retry the interaction once the element becomes stable again

Handling Stale Element References:

Stale element references occur when the DOM changes after an element is located. To address this, testers can re-query the DOM for the element before attempting to interact with it. Alternatively, testers can implement a retry mechanism within the script that retries the interaction after catching a stale element reference exception. This approach ensures that tests do not fail due to transient changes in the DOM.

Page Object Model (POM):

The Page Object Model (POM) is a design pattern that promotes the creation of modular and maintainable test scripts. It encapsulates the logic for locating and interacting with **elements** within a class that represents the **web** page, rather than directly coding these interactions into the test case itself. This abstraction makes it easier to update locators when the underlying page structure changes, as the changes need to be made only in the page object class. The POM, when combined with advanced locators and wait strategies, creates a robust foundation for managing **dynamic elements** across different pages and scenarios.

Third-Party Libraries and Tools:

Selenium's functionality can be enhanced by integrating it with third-party tools and libraries, improving its ability to handle **dynamic** content. Tools like Sikuli, which enables image-based automation and Applitools, which performs visual validation, can complement **Selenium's** DOM-based interactions. Additionally, integrating **Selenium** with test frameworks such as TestNG or JUnit can improve test organization, execution, and reporting, while **handling** retries and parallel execution more effectively.

CASE STUDIES

Handling Dynamic Dropdowns in AngularJS Applications:

In a real-world scenario involving an AngularJS-based e-commerce application, the development team encountered a **dynamic** dropdown that was populated asynchronously based on user input. The dropdown was not present in the DOM until an AJAX request completed and updated the page content. To automate this interaction, the team utilized relative XPath expressions to locate the dropdown based on nearby static **elements**. Additionally, explicit waits were employed to ensure that the dropdown was fully loaded before the script attempted to select an option. This combination of techniques allowed the team to create a robust and reliable test that interacted with **dynamic** dropdowns across different browsers and environments.

Cross-Browser Testing in React-Based Web Applications:

A banking **web** application built with React posed significant challenges for the automation team due to its heavy

reliance on **dynamic elements**. **Elements** such as buttons and form fields were loaded asynchronously and often appeared at different times depending on the browser and network conditions. In Chrome, **elements** typically loaded faster, while Firefox and Internet Explorer required longer waits. The team employed browser-specific wait times, along with the Page Object Model, to ensure consistent interactions across all supported browsers. By implementing fluent waits and cross-browser testing strategies, the team was able to maintain stable test results, even as the underlying **web** application continued to evolve.

Fig.4 *Comparison of Wait Strategies Across Browsers*

BEST PRACTICES FOR HANDLING DYNAMIC ELEMENTS

To effectively handle **dynamic web elements** in **Selenium**, it is essential to follow best practices that maximize script reliability and minimize test maintenance.

• **Use Stable Locators:** Avoid using locators that are prone to frequent changes, such as autogenerated IDs or dynamically generated class names. Instead, use more reliable locator strategies, such as relative XPath or CSS selectors based on stable attributes or relationships between **elements**.

• **Leverage Explicit and Fluent Waits:** Always use explicit or fluent waits to handle **dynamic** content and asynchronous loading. These waits ensure that **elements** are fully loaded and ready for interaction, preventing common errors such as "element not found" or "stale element reference."

• **Modularize Test Scripts with POM:** Implement the Page Object Model to encapsulate page-specific logic and interactions within a reusable class structure. This makes it easier to maintain tests when the underlying page structure changes and promotes code reuse across multiple test cases.

• **Cross-Browser Testing:** Regularly run tests across different browsers to identify and resolve cross-browser inconsistencies in the way **dynamic elements** are rendered and interacted with. Browser-specific waits, locators, or actions may be necessary to achieve consistent behavior.

• **Handle Stale Element References:**

Implement retry mechanisms to handle stale element references caused by changes in the DOM. This can significantly reduce the number of test failures caused by transient DOM updates.

Fig.5 *Performance Improvement of Automation Scripts*

CONCLUSION

Dynamic web elements, which are increasingly common in modern **web** applications, present significant challenges for automation testing. **Selenium**, while a powerful and versatile tool, requires a strategic approach to reliably handle **dynamic** content. By employing advanced locators such as XPath and CSS selectors, using explicit and

fluent waits, modularizing test scripts with the Page Object Model, and integrating third-party tools and libraries, testers can create robust and maintainable automation scripts that perform consistently across different browsers and environments.

The techniques and best practices outlined in this paper provide a framework for building resilient automation scripts capable of **handling** the **dynamic** behaviors inherent in today's **web** applications. As **web** technologies continue to evolve, these strategies will remain essential for ensuring the reliability and scalability of automated testing efforts.

REFERENCES

[1]. Leotta, M., Clerissi, D., Ricca, F., & Spadaro, C. (2013). Repairing **Selenium** Test Cases: An Industrial Case Study about **Web** Page Element Localization. 2013 IEEE Sixth International Conference on Software Testing, Verification and Validation, 487-488. https://doi.org/10.1109/ICST.2013.73.

[2]. Berner, S., Weber, R., & Keller, R. (2005). Observations and lessons learned from automated testing. Proceedings. 27th International Conference on Software Engineering, 2005. ICSE 2005., 571-579. https://doi.org/10.1145/1062455.1062556.

[3]. Haugset, B., & Hanssen, G. (2008). Automated Acceptance Testing: A Literature Review and an Industrial Case Study. Agile 2008 Conference, 27-38. https://doi.org/10.1109/Agile.2008.82.

[4]. Page Object Model (POM) Design Pattern, Using the Page Object Model in **Selenium** Automation Scripts. Available at: https://www.**selenium**.dev/documentation/en/guidelines_and_recommendations/page_object_models/

[5]. XPath and CSS Selectors in **Selenium**, Best Practices for Locating **Elements** in **Selenium** Using XPath and CSS. Available at: https://www.**selenium**.dev/documentation/webdriver/**elements**/finders/

Page 6

[6]. Zhao-shan, Y. (2009). Automated testing framework based on XML with level and weight attributes. Journal of Hefei University of Technology. https://www.semanticscholar.org/paper/Automated-testing-framework-based-on-XML-with-level-Zhao-shan/aee7fd046bf42b3e2b591ce56acd2aea0c3c88b5

[7]. Garousi, V., & Mäntylä, M. (2016). When and what to automate in software testing? A multi-vocal literature review. Inf. Softw. Technol., 76, 92-117. https://doi.org/10.1016/j.infsof.2016.04.015.

[8]. Subramanyan, R. (2007). Test Automation in Practice. 31st Annual International Computer Software and Applications Conference (COMPSAC 2007), 1, 19-19. https://doi.org/10.1109/COMPSAC.2007.207.

[9]. Collins, E., & Lucena, V. (2012). Software Test Automation practices in agile development environment: An industry experience report. 2012 7th International Workshop on Automation of Software Test (AST), 57-63. https://doi.org/10.5555/2663608.2663620.

[10]. Alsmadi, I. (2012). Advanced Automated Software Testing: Frameworks for Refined Practice. . https://doi.org/10.4018/978-1-46660-089-8.

[11]. Budnik, C., Fraser, G., Lonetti, F., & Zhu, H. (2018). Special issue on automation of software testing: improving practical applicability. Software Quality Journal, 26, 1415-1419. https://doi.org/10.1007/s11219-018-9410-1.